

The Math Behind Our ML Implementations

A first-principles companion to the NumPy implementations we built.

This is not a general mathematics textbook. Every mathematical idea here is tied directly to something we actually used: linear regression, logistic regression, neural networks, MNIST, loss functions, backpropagation, and gradient descent.

How to use this PDF: Don't memorize formulas. For each one, understand (1) what quantity we are calculating, (2) where the expression comes from, (3) what it tells the model, and (4) where it appears in our NumPy code.

1. The basic model: $\hat{y} = wx + b$

Our first model was linear regression. We did not give the computer the rule $\hat{y} = 2x + 1$. We gave it examples of x and y and let it learn w and b .

What the equation means

For one input x , the model calculates a prediction $\hat{y}$. The weight w controls how strongly the input affects the prediction, while b shifts the line up or down.

$$\hat{y} = wx + b$$

Why w is the slope

If x increases by Δx , the prediction changes by $w\Delta x$. Therefore the prediction change per unit change in x is w .

$$\Delta\hat{y} = w\Delta x \rightarrow \Delta\hat{y} / \Delta x = w$$

Why b is the intercept

If $x = 0$, the input contribution disappears and only b remains.

$$\hat{y} = w(0) + b = b$$

Why this matters for learning

The model initially has the wrong w and b . Training means repeatedly changing them so the predictions become closer to the known y values.

2. Prediction error and Mean Squared Error

The model needs a numerical way to measure how far its prediction is from the correct answer.

Error

For one example, prediction error is simply prediction minus the target.

$$\text{error} = \hat{y} - y$$

Why square the error?

If we simply averaged signed errors, positive and negative mistakes could cancel. Squaring also makes larger mistakes count more strongly.

$$\text{error}^2 = (\hat{y} - y)^2$$

Mean Squared Error

For N examples, calculate every squared error and average them.

$$L = (1/N) \sum_i (\hat{y}_i - y_i)^2$$

$$\text{loss} = \text{np.mean}((y_{\text{pred}} - y) ** 2)$$

Why the mean?

Averaging keeps the scale of the loss from depending directly on how many examples are in the batch.

3. Why derivatives appeared in training

A loss value alone does not tell us how to change w or b . We need to know how the loss changes when a parameter changes.

The question a derivative answers

For w , we ask: if I make w slightly larger, does the loss increase or decrease, and by roughly how much?

$$\partial L / \partial w$$

Deriving the linear-regression gradient

Start with the exact path from w to the loss: w changes the prediction; the prediction changes the error; the error determines the squared loss.

$$w \rightarrow \hat{y} = wx + b \rightarrow \text{error} = \hat{y} - y \rightarrow L = (\hat{y} - y)^2$$

Step 1: derivative of loss with respect to prediction

Differentiate the square. The exponent 2 comes down and the inside remains.

$$\partial L / \partial \hat{y} = 2(\hat{y} - y)$$

Step 2: derivative of prediction with respect to w

In $\hat{y} = wx + b$, changing w changes wx in proportion to x .

$$\partial \hat{y} / \partial w = x$$

Step 3: connect the two using the chain rule

Because w affects L through $\hat{y}$, multiply the two local effects.

$$\partial L / \partial w = (\partial L / \partial \hat{y})(\partial \hat{y} / \partial w)$$

$$\partial L / \partial w = 2(\hat{y} - y)x$$

For the whole dataset

Because our loss is the average of N squared errors, we average the individual gradients too.

$$\partial L / \partial w = (2/N) \sum_i (\hat{y}_i - y_i)x_i$$

$$\partial L / \partial b = (2/N) \sum_i (\hat{y}_i - y_i)$$

```
error = y_pred - y
dw = (2/N) * X.T @ error
db = (2/N) * np.sum(error)
```

4. Gradient descent: turning the derivative into learning

Once we know the direction in which the loss changes, we move the parameter in the opposite direction.

Why opposite?

A positive gradient means increasing the parameter increases the loss. To reduce the loss, move the parameter downward. A negative gradient means increasing it would reduce the loss.

$$w_{\text{new}} = w - \eta(\partial L / \partial w)$$

Learning rate

η (eta) is the step size. The gradient gives the local direction and magnitude of change; the learning rate controls how much of that suggested step we take.

$$w = w - \text{lr} * dw$$

$$b = b - lr * db$$

5. Sigmoid: turning a score into a 0-1 value

For binary classification, a linear score can be any real number. We wanted a value that behaves like a probability, so we introduced sigmoid.

The function

Sigmoid takes any real number z and smoothly maps it between 0 and 1.

$$\sigma(z) = 1 / (1 + e^{(-z)})$$

Why the output behaves this way

When z is very positive, $e^{(-z)}$ becomes very small, so $\sigma(z)$ approaches 1. When z is very negative, $e^{(-z)}$ becomes very large, so $\sigma(z)$ approaches 0. At $z = 0$, the denominator is 2.

$$\sigma(0) = 1 / (1 + 1) = 0.5$$

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))
```

6. Binary cross-entropy: why the logarithm appears

For logistic regression, the model outputs a probability. We need a loss that strongly penalizes being confidently wrong.

When the true label is 1

We want the loss to be small when the predicted probability p is near 1 and very large when p is near 0.

$$L = -\log(p)$$

Why log gives the desired behavior

For p close to 1, $\log(p)$ is close to 0, so $-\log(p)$ is small. As p approaches 0, $\log(p)$ becomes very negative, so $-\log(p)$ becomes very large.

$$p = 0.9 \rightarrow L \approx 0.105 \quad p = 0.5 \rightarrow L \approx 0.693 \quad p = 0.1 \rightarrow L \approx 2.303$$

For label 0

If the true label is 0, the desired probability is $1 - p$.

$$L = -\log(1 - p)$$

Combine both cases

Because y is either 0 or 1, the expression automatically selects the appropriate case.

$$L = -[y \log(p) + (1-y) \log(1-p)]$$

7. Why logistic regression gets the simple gradient $p - y$

This is the result we used in code. The derivative becomes simple because sigmoid and binary cross-entropy combine neatly.

The path

The model is $z = wx + b$, then $p = \text{sigmoid}(z)$, then BCE produces L .

$$w \rightarrow z \rightarrow p \rightarrow L$$

Sigmoid derivative

Starting from $\sigma(z) = 1/(1+e^{(-z)})$, differentiation simplifies to:

$$dp/dz = p(1 - p)$$

BCE derivative with respect to p

For $L = -[y \log(p) + (1-y)\log(1-p)]$, differentiation gives:

$$dL/dp = -y/p + (1-y)/(1-p)$$

Multiply through the chain

Applying $dL/dz = (dL/dp)(dp/dz)$ and simplifying gives:

$$dL/dz = p - y$$

Then reach w and b

Because $z = wx + b$, $dz/dw = x$ and $dz/db = 1$.

$$dL/dw = (p - y)x$$

$$dL/db = p - y$$

```
error = y_pred - y
dw = (1/N) * X.T @ error
db = (1/N) * np.sum(error)
```

Why there was no 2 here

Linear regression used squared error, whose derivative produces the factor 2. BCE combined with sigmoid simplifies to $p - y$, so there is no factor 2.

8. Vectors and matrices: why neural networks use them

A neuron has many inputs and many weights. Vectors and matrices let us express all those multiplications compactly.

One neuron

With inputs x_1, x_2, x_3 and weights w_1, w_2, w_3 :

$$z = x_1w_1 + x_2w_2 + x_3w_3 + b$$

Vector form

Put inputs and weights into vectors. Their dot product performs the sum of pairwise products.

$$z = x \cdot w + b$$

Many examples and neurons

For MNIST, a batch of 32 images has shape (32,784), while 128 hidden neurons require a weight matrix of shape (784,128).

$$(32 \times 784)(784 \times 128) = (32 \times 128)$$

$$Z1 = X @ W1 + b1$$

Why dimensions have to match

Each hidden neuron needs one weight for every input pixel. Therefore W1 has 784 rows and 128 columns.

9. Transpose: why X^T appears in backpropagation

We used transposes such as $X^T dZ$ when calculating weight gradients. This follows directly from the dimensions and the connection structure.

The shapes

For a batch:

$$X: (32, 784) \quad dZ: (32, 128)$$

We need the shape of W1's gradient

W1 has shape (784,128), so dW1 must have the same shape. Transposing X gives:

$$X^T: (784, 32)$$

$$(784 \times 32)(32 \times 128) = (784 \times 128)$$

The underlying calculation

For input feature j and neuron k, the gradient adds the contribution from every example:

$$dW[j,k] = \sum_i X[i,j] \cdot dZ[i,k]$$

10. ReLU and why its derivative is so simple

Our hidden layer uses ReLU to introduce non-linearity.

$$\text{ReLU}(z) = \max(0, z)$$

What it does

Positive values pass through unchanged. Negative values become zero.

$$z = -2 \rightarrow \text{ReLU}(z) = 0 \quad z = 3 \rightarrow \text{ReLU}(z) = 3$$

Derivative

For positive inputs, changing z changes the output one-for-one. For negative inputs, the output is flat at zero.

$$\text{ReLU}'(z) = 1 \text{ if } z > 0 \quad 0 \text{ if } z \leq 0$$

$$dZ1 = dA1 * (Z1 > 0)$$

11. Backpropagation: assembling the chain rule

Backpropagation is the chain rule applied systematically backward through our computational graph.

Our two-layer network

The forward dependencies are:

$$X \rightarrow Z1 \rightarrow A1 \rightarrow Z2 \rightarrow \text{Loss}$$

Start at the loss

For MNIST, softmax plus cross-entropy gives the output gradient:

$$dZ2 = (P - Y) / N$$

Move one layer backward

Because $Z2 = A1W2 + b2$:

$$dW2 = A1^T dZ2 \quad db2 = \sum_i dZ2[i,:]$$

Continue backward

The gradient with respect to $A1$ is passed through $W2$:

$$dA1 = dZ2 W2^T$$

Pass through ReLU

ReLU either passes the gradient or blocks it:

$$dZ1 = dA1 \odot \text{ReLU}'(Z1)$$

Reach the first layer's parameters

Since $Z1 = XW1 + b1$:

$$dW1 = X^T dZ1 \quad db1 = \sum_i dZ1[i,:]$$

The key idea

Every gradient answers a local question: how does this quantity affect the quantity immediately after it? Multiplying those local effects connects the whole path back to the loss.

12. Softmax: deriving the output probabilities

MNIST has 10 mutually exclusive classes. We want 10 scores to become a probability distribution.

Exponentiate the logits

Exponentials make every score positive and amplify differences between scores.

$$e^{(z_i)}$$

Normalize them

To make the values add up to one, divide each exponential by the sum of all exponentials.

$$P_i = e^{(z_i)} / \sum_j e^{(z_j)}$$

Why the probabilities sum to 1

Adding every P_i uses the same denominator:

$$\sum_i P_i = \sum_i e^{(z_i)} / \sum_j e^{(z_j)} = 1$$

```
def softmax(z):  
    z = z - np.max(z, axis=1, keepdims=True)  
    exp_z = np.exp(z)  
    return exp_z / np.sum(exp_z, axis=1, keepdims=True)
```

Why subtract the maximum?

Subtracting the same value from every logit does not change the final probabilities because the common factor cancels. It prevents exponentials from becoming unnecessarily huge.

13. The MNIST batch equations as one mathematical system

All the pieces now fit together.

$$Z1 = XW1 + b1$$

$$A1 = \text{ReLU}(Z1)$$

$$Z2 = A1W2 + b2$$

$$P = \text{Softmax}(Z2)$$

$$L = -(1/N) \sum_i \log(P[i, y_i])$$

$$dZ2 = (P - Y)/N$$

$$dW2 = A1^T dZ2$$

$$db2 = \sum_i dZ2[i,:]$$

$$dA1 = dZ2 W2^T$$

$$dZ1 = dA1 \odot \text{ReLU}'(Z1)$$

$$dW1 = X^T dZ1$$

$$db1 = \sum_i dZ1[i,:]$$

Gradient descent updates the parameters

Every learned parameter moves opposite its gradient.

$$W1 \leftarrow W1 - \eta dW1 \quad b1 \leftarrow b1 - \eta db1 \quad W2 \leftarrow W2 - \eta dW2 \quad b2 \leftarrow b2 - \eta db2$$

14. What each piece means in our code

This mapping is useful when reading the notebook.

Math	NumPy
$\hat{y} = XW + b$	<code>X @ W + b</code>
MSE	<code>np.mean((y_pred-y)**2)</code>
Sigmoid	<code>1/(1+np.exp(-z))</code>
BCE	<code>-np.mean(y*np.log(p)+(1-y)*np.log(1-p))</code>
ReLU	<code>np.maximum(0, z)</code>
Softmax	<code>exp(z)/sum(exp(z))</code>
Correct class probability	<code>probabilities[np.arange(N), y]</code>
Gradient	<code>dW, db, dZ, dA</code>
Gradient descent	<code>W -= lr * dW</code>
Matrix multiplication	<code>@</code>
Transpose	<code>.T</code>

15. The mental model to keep

The most useful way to remember this mathematics is not as a list of formulas.

Forward: parameters → prediction → loss

Backward: loss → gradients → parameters

Forward asks

Given the parameters I currently have, what prediction do they produce?

Loss asks

How bad was that prediction compared with the known answer?

Backward asks

Which parameters contributed to that error, and in what direction?

Update asks

How should I change those parameters to reduce the error?

Repeat

Training repeats this process over many examples until the learned parameters produce useful predictions.

Scope note: This document intentionally stops at the mathematics we have actually encountered in our implementations. More advanced mathematics can be added when we encounter it in the learning path.